

Project Explanation

1. Problem approach

I treated this as a simple, production-minded audio transcription pipeline with two main goals:

- Make audio format handling robust by normalizing inputs once.
- Make transcription reusable and testable with both real and mock backends.

The repo is structured so the CLI, web UI, and pipeline all reuse the same core logic in `transcription_pipeline/`.

2. Key design decisions

A. Normalize every audio file up front

File: `transcription_pipeline/audio_utils.py`

- `probe_audio()` uses `ffprobe` to inspect the input audio stream.
- `normalize_to_wav()` converts any supported container/codecs into:
 - mono
 - 16 kHz
 - PCM WAV

This means the rest of the system only needs to handle one canonical format.

B. Handle long audio with chunking

File: `transcription_pipeline/audio_utils.py`

- `iter_chunks()` splits long audio into overlapping chunks
- Each chunk is transcribed independently and deleted immediately
- The overlap protects against words cut off at chunk boundaries
- The pipeline later removes duplicate overlapped segments

C. Separate backend interface

File: `transcription_pipeline/backends.py`

- `TranscriptionBackend` is an abstract interface
- `FasterWhisperBackend` is the real model backend
- `MockBackend` is a deterministic fake backend for tests / demos

D. Structured outputs for reuse

File: `transcription_pipeline/postprocess.py`

- TranscriptResult can emit JSON, SRT, and plain TXT
- top_keywords() provides a simple keyword extraction example

3. How the code works (core flow)

File: transcription_pipeline/pipeline.py

TranscriptionPipeline.run() performs:

1. probe_audio() — inspect file
2. normalize_to_wav() — convert to 16kHz mono WAV
3. iter_chunks() — generate overlapping chunks if needed
4. backend.transcribe() — transcribe each chunk
5. offset segment timestamps by chunk start
6. _dedupe_overlapping_segments() — remove duplicated overlap
7. build TranscriptResult and write outputs if requested

4. Testing and readiness

- tests/ uses MockBackend so test runs are fast and deterministic.
- pytest in your environment passed: 14 passed.
- README.md includes quickstart and web UI instructions.

5. Practical notes for the client

- ffmpeg must be installed and available on PATH.
- Real transcription requires faster-whisper (listed in requirements.txt).
- Web UI is a demo: consider adding auth, upload cleanup, and production deployment plumbing before public deployment.

If you want the PDF formatted differently (cover page, logo, or code snippets embedded), tell me how and I will update it.